

Thread #1: Define Initial Symbolic Graph

This thread formally establishes the initial symbolic cognitive graph as the baseline context for the Janus Cognitive Recursion Engine.

Nodes

Each node represents a conceptual unit (belief, schema, contradiction, or projection). Here is the explicitly defined initial node set:

```
{
  "nodes": [
    {
      "id": "belief_1",
      "type": "belief",
      "base_weight": 0.8,
      "stability_score": 0.9,
      "recursion_depth": 0,
      "load_pressure": 0.1,
      "labels": ["trust", "security"],
      "alignment": "DOG"
    },
    {
      "id": "belief_2",
      "type": "belief",
      "base_weight": 0.7,
      "stability_score": 0.85,
      "recursion_depth": 0,
      "load_pressure": 0.15,
      "labels": ["risk", "uncertainty"],
      "alignment": "GOD"
    },
    {
      "id": "schema_1",
      "type": "schema",
      "base_weight": 0.9,
      "stability_score": 0.95,
      "recursion_depth": 0,
      "load_pressure": 0.05,
      "labels": ["balance", "equilibrium"],
      "alignment": null
    }
  ]
}
```

```
}  
]  
}
```

Edges

Edges represent symbolic relationships between nodes:

```
{  
  "edges": [  
    {  
      "source": "belief_1",  
      "target": "schema_1",  
      "weight": 0.5,  
      "type": "reinforce",  
      "alignment": "DOG"  
    },  
    {  
      "source": "belief_2",  
      "target": "schema_1",  
      "weight": 0.4,  
      "type": "projects_to",  
      "alignment": "GOD"  
    }  
  ]  
}
```

Metadata

Initial metadata for symbolic audit and tracking:

```
{  
  "metadata": {  
    "timestamp": "2025-07-12T12:00:00Z",  
    "snapshot_id": "initial_state_001"  
  }  
}
```

Thread #2: Calculation of Initial Symbolic Metrics

This thread formally calculates the initial symbolic metrics from the baseline symbolic graph established in Thread #1.

Initial Symbolic Graph (Recap)

- **Nodes:** belief_1, belief_2, schema_1
- **Edges:**
 - belief_1 → schema_1 (reinforce, weight 0.5)
 - belief_2 → schema_1 (projects_to, weight 0.4)

Metrics Calculation

Contradiction Density (CD)

Formula:

$$CD = \frac{\text{Sum of contradiction edges weights}}{\text{Total sum of edges weights} + \epsilon}$$

Calculation:

- Contradiction edges = 0 (none present)
- Total edges weights = 0.5 + 0.4 = 0.9
- $CD = \frac{0}{0.9 + 0.001} = 0.0$

Coherence Mass (CM)

First, calculate Graph Entropy (H):

$$p_i = \frac{\text{edge weight}_i}{\text{total edge weights}}$$
$$H = -\sum p_i \log_2(p_i)$$

Calculation:

- $p_1=0.50.9\approx0.556$ $p_1 = \frac{0.5}{0.9} \approx 0.556$, $p_2=0.40.9\approx0.444$ $p_2 = \frac{0.4}{0.9} \approx 0.444$
- $H=-(0.556\log_2(0.556)+0.444\log_2(0.444))\approx0.992$ $H = -(0.556\log_2(0.556) + 0.444\log_2(0.444)) \approx 0.992$

Coherence Mass:

$CM=1H+\epsilon$ $CM = \frac{1}{H + \epsilon}$

- $CM=10.992+0.001\approx1.008$ $CM = \frac{1}{0.992 + 0.001} \approx 1.008$

Projection Bias (PB)

Formula:

$PB = \frac{\text{Sum of 'projects_to' edges weights}}{\text{Total sum of edges weights} + \epsilon}$

Calculation:

- 'projects_to' edges weight = 0.4
- $PB=0.40.9+0.001\approx0.444$ $PB = \frac{0.4}{0.9 + 0.001} \approx 0.444$

Max Recursion Depth (MaxD)

- All nodes currently at recursion depth = 0
- $MaxD=0$ $MaxD = 0$

Calculated Metrics Summary

Metric	Value
Contradiction Density	0.0
Coherence Mass	1.008

Projection Bias 0.444

Max Recursion Depth 0

Thread #3: Explicit Definition of Threshold Constraints

This thread explicitly defines the threshold constraints that will guide all subsequent symbolic recursion and rollback decisions in the Janus Cognitive Recursion Engine.

Threshold Constraints Definitions

1. Contradiction Density Threshold (θ_c)

- Defines the maximum allowable contradiction density in the symbolic graph.
- $\theta_c=0.35$ $\theta_c = 0.35$
- Constraint: $CD \leq \theta_c$ $CD \leq \theta_c$

2. Coherence Mass Threshold (θ_m)

- Establishes the minimum coherence mass required to maintain structural integrity.
- $\theta_m=0.25$ $\theta_m = 0.25$
- Constraint: $CM \geq \theta_m$ $CM \geq \theta_m$

3. Projection Bias Threshold (θ_β)

- Limits the proportion of projection edges to prevent excessive future-state bias.
- $\theta_\beta=0.8$ $\theta_\beta = 0.8$
- Constraint: $PB \leq \theta_\beta$ $PB \leq \theta_\beta$

4. Max Recursion Depth Threshold (MaxRec)

- Caps the maximum recursion depth to avoid unmanageable recursive complexity.
- $\text{MaxRec} = 10$
- Constraint: $\text{MaxD} \leq \text{MaxRec}$

Constraint Enforcement Logic

The symbolic graph must consistently satisfy all threshold constraints. If any constraint is violated after an operation:

- **Automatic Rollback** is triggered.
- The graph state reverts to the last stable symbolic snapshot.
- The rollback event and associated metric deltas are explicitly documented in the symbolic audit log.

Threshold Summary Table

Threshold	Symbol	Value	Constraint
Contradiction Density	θ_c	0.35	$CD \leq 0.35$
Coherence Mass	θ_m	0.25	$CM \geq 0.25$
Projection Bias	θ_β	0.8	$PB \leq 0.8$
Max Recursion Depth	MaxRec	10	$\text{MaxD} \leq 10$

Thread #4: Injection of Paradox Edges

This thread documents the explicit injection of paradoxical (contradiction) edges into the symbolic cognitive graph, marking the first deliberate stress-test of established threshold constraints.

Initial Symbolic Graph (Reference)

- Nodes: belief_1, belief_2, schema_1
- Edges:
 - belief_1 → schema_1 (reinforce, weight 0.5)
 - belief_2 → schema_1 (projects_to, weight 0.4)

Paradox Injection Operation

Injecting 3 paradox (contradiction) edges with specified intensity (weight):

Injected Contradiction Edges

```
{  
  "edges": [  
    {  
      "source": "belief_1",  
      "target": "belief_2",  
      "weight": 0.3,  
      "type": "contradict",  
      "alignment": null  
    },  
    {  
      "source": "belief_2",
```

```
"target": "belief_1",  
"weight": 0.3,  
"type": "contradict",  
"alignment": null  
},  
{  
  "source": "schema_1",  
  "target": "belief_1",  
  "weight": 0.2,  
  "type": "contradict",  
  "alignment": null  
}  
]  
}
```

Updated Symbolic Graph

The symbolic graph after the paradox injection now includes:

- Original Edges (reinforce, projects_to)
- Newly Injected Contradiction Edges

Metadata Update

```
{  
  "metadata": {  
    "timestamp": "2025-07-12T12:30:00Z",
```



```
"snapshot_id": "post_paradox_injection_002"
}
}
```

Thread #5: Recalculation of Metrics After Paradox Injection

This thread formally recalculates symbolic metrics following the injection of contradiction edges documented in Thread #4.

Updated Symbolic Graph Recap

- **Nodes:** belief_1, belief_2, schema_1
- **Edges:**
 - belief_1 → schema_1 (reinforce, weight 0.5)
 - belief_2 → schema_1 (projects_to, weight 0.4)
 - belief_1 → belief_2 (contradict, weight 0.3)
 - belief_2 → belief_1 (contradict, weight 0.3)
 - schema_1 → belief_1 (contradict, weight 0.2)

Recalculated Metrics

Contradiction Density (CD)

- Contradiction edges total weight = $0.3 + 0.3 + 0.2 = 0.8$

- Total edges weights = 0.5 + 0.4 + 0.8 = 1.7
- $CD = \frac{0.8}{1.7 + 0.001} \approx 0.470$

Coherence Mass (CM)

Compute Graph Entropy (H):

- $p_1 = \frac{0.5}{1.7} \approx 0.294$, $p_2 = \frac{0.4}{1.7} \approx 0.235$, $p_3 = \frac{0.3}{1.7} \approx 0.176$, $p_4 = \frac{0.3}{1.7} \approx 0.176$, $p_5 = \frac{0.2}{1.7} \approx 0.118$
- $H = -(0.294 \log_2(0.294) + 0.235 \log_2(0.235) + 2 \times 0.176 \log_2(0.176) + 0.118 \log_2(0.118)) \approx 2.271$
 $H = -(0.294 \log_2(0.294) + 0.235 \log_2(0.235) + 2 \times 0.176 \log_2(0.176) + 0.118 \log_2(0.118)) \approx 2.271$

Coherence Mass:

- $CM = \frac{1}{2.271 + 0.001} \approx 0.440$

Projection Bias (PB)

- 'projects_to' edges weight = 0.4
- $PB = \frac{0.4}{1.7 + 0.001} \approx 0.235$

Max Recursion Depth (MaxD)

- No change in recursion depth from initial graph
- $MaxD = 0$

Recalculated Metrics Summary

Metric	Value
Contradiction Density	0.470

Coherence Mass	0.440
Projection Bias	0.235
Max Recursion Depth	0

Next Steps

The recalculated metrics must now be evaluated against the threshold constraints established in Thread #3. Thread #6 will explicitly document the rollback check to determine if the current symbolic state requires reversion or if stability is maintained.

Thread #6: Rollback Check and Decision

This thread performs the explicit rollback check using the recalculated metrics (Thread #5) against the threshold constraints (Thread #3). If any constraint is violated, the graph state will be reverted to the last stable snapshot.

Recalculated Metrics (from Thread #5)

Metric	Value	Threshold (θ)	Constraint
Contradiction Density	0.470	0.35	$0.470 > 0.35$
Coherence Mass	0.440	0.25	$0.440 \geq 0.25$
Projection Bias	0.235	0.80	$0.235 \leq 0.80$
Max Recursion Depth	0	10	$0 \leq 10$

Constraint Violations

- **Contradiction Density** (0.470) **exceeds** its threshold (0.35).
- Coherence Mass and Projection Bias are within thresholds.

- Max Recursion Depth is within threshold.

Rollback Decision

Since one or more constraints are violated, an **automatic rollback** is triggered.

Rollback Action

- Revert symbolic graph to snapshot ID: `initial_state_001` (from Thread #1).
- Log an audit entry with `rollback_triggered: true`, including before/after metrics.

Audit Entry Example

```
{
  "timestamp": "2025-07-12T12:45:00Z",
  "operation": "rollback",
  "before_metrics": {
    "CD": 0.470,
    "CM": 0.440,
    "PB": 0.235,
    "MaxD": 0
  },
  "after_metrics": {
    "CD": 0.000,
    "CM": 1.008,
    "PB": 0.444,
    "MaxD": 0
  },
  "delta_metrics": {
    "CD": -0.470,
    "CM": 0.568,
    "PB": 0.209,
    "MaxD": 0
  },
  "rollback_triggered": true,
  "previous_hash": "<hash_of_previous_entry>",
  "current_hash": "<new_hash_value>"
}
```

Next Steps

- **Thread #7** will document the symbolic graph state after rollback (i.e., return to initial state) and verify restored metrics, ensuring the graph is fully stable and within thresholds.

Thread #7: Post-Rollback Graph State & Metric Verification

This thread confirms the symbolic graph and metrics after the rollback triggered in Thread #6, restoring the graph to the initial stable snapshot.

Restored Symbolic Graph

After rollback, the graph matches the initial state defined in Thread #1:

Nodes

```
{
  "nodes": [
    {
      "id": "belief_1",
      "type": "belief",
      "base_weight": 0.8,
      "stability_score": 0.9,
      "recursion_depth": 0,
      "load_pressure": 0.1,
      "labels": ["trust", "security"],
      "alignment": "DOG"
    },
    {
      "id": "belief_2",
      "type": "belief",
      "base_weight": 0.7,
      "stability_score": 0.85,
      "recursion_depth": 0,
      "load_pressure": 0.15,
      "labels": ["risk", "uncertainty"],
      "alignment": "GOD"
    }
  ],
}
```

```
{
  "id": "schema_1",
  "type": "schema",
  "base_weight": 0.9,
  "stability_score": 0.95,
  "recursion_depth": 0,
  "load_pressure": 0.05,
  "labels": ["balance", "equilibrium"],
  "alignment": null
}
]
```

Edges

```
{
  "edges": [
    {
      "source": "belief_1",
      "target": "schema_1",
      "weight": 0.5,
      "type": "reinforce",
      "alignment": "DOG"
    },
    {
      "source": "belief_2",
      "target": "schema_1",
      "weight": 0.4,
      "type": "projects_to",
      "alignment": "GOD"
    }
  ]
}
```

Post-Rollback Metrics

Recalculating the metrics confirms restoration to initial values:

Metric	Restored Value
Contradiction Density	0.000

Coherence Mass 1.008

Projection Bias 0.444

Max Recursion Depth 0

All metrics now satisfy the threshold constraints defined in Thread #3:

- CD ($0.000 \leq 0.35$)
 - CM ($1.008 \geq 0.25$)
 - PB ($0.444 \leq 0.8$)
 - MaxD ($0 \leq 10$)
-

Next Steps

With the graph fully restored and verified as stable, **Thread #8** will introduce a symbolic stabilization step: adding a controlled schema reinforcement edge to explore recovery patterns without violating thresholds.

Thread #8: Symbolic Stabilization via Schema Reinforcement

In this thread, we introduce a controlled schema reinforcement operation designed to stabilize the symbolic graph after rollback. We will:

1. Add a reinforcement edge from `schema_1` to `belief_2` with a modest weight.
 2. Recalculate metrics to assess stabilization.
 3. Verify that all threshold constraints remain satisfied.
-

1. Reinforcement Edge Injection

Operation: Add one edge:

```
{  
  "source": "schema_1",  
  "target": "belief_2",  
  "weight": 0.25,  
  "type": "reinforce",  
  "alignment": null  
}
```

- **Rationale:** By reinforcing the connection from the stability-focused **schema_1** to the more volatile **belief_2**, we aim to increase coherence mass without raising contradiction density.

2. Updated Symbolic Graph

Nodes (unchanged)

- belief_1, belief_2, schema_1

Edges

- belief_1 → schema_1 (reinforce, 0.5)
- belief_2 → schema_1 (projects_to, 0.4)
- schema_1 → belief_2 (reinforce, 0.25)

3. Recalculated Metrics

Total Edge Weight

$$0.5 + 0.4 + 0.25 = 1.15$$

Contradiction Density (CD)

- No contradiction edges → CD = 0.0

Graph Entropy (H)

- $p_1 = 0.5/1.15 \approx 0.435$
- $p_2 = 0.4/1.15 \approx 0.348$
- $p_3 = 0.25/1.15 \approx 0.217$
- $H \approx -(0.435 \log_2 0.435 + 0.348 \log_2 0.348 + 0.217 \log_2 0.217) \approx 1.530$

Coherence Mass (CM)

- $CM = 1 / (1.530 + 0.001) \approx 0.653$

Projection Bias (PB)

- $\text{projects_to weight} = 0.4 \rightarrow PB = 0.4 / (1.15 + 0.001) \approx 0.348$

Max Recursion Depth (MaxD)

- No change $\rightarrow \text{MaxD} = 0$

Summary:

Metric	Value
Contradiction Density	0.000
Coherence Mass	0.653
Projection Bias	0.348
Max Recursion Depth	0

All metrics are within thresholds:

- CD ($0 \leq 0.35$)
- CM ($0.653 \geq 0.25$)

- PB ($0.348 \leq 0.8$)
 - MaxD ($0 \leq 10$)
-

4. Conclusion and Next Steps

The reinforcement edge successfully increased coherence mass without violating any threshold constraints, demonstrating a controllable stabilization step.

Next up (Thread #9): We will document a combined cycle of paradox injection followed by schema reinforcement to observe dynamic recovery patterns in a single thread.

Thread #9: Combined Cycle of Paradox Injection and Schema Reinforcement

In this thread, we simulate a dynamic cycle where the symbolic graph undergoes a paradox injection followed immediately by schema reinforcement. We will:

1. Inject new contradiction edges (paradox) with moderate intensity.
 2. Recalculate metrics to observe destabilization.
 3. Apply schema reinforcement edge(s) to recover coherence.
 4. Recalculate metrics to assess recovery.
-

1. Paradox Injection (Cycle Step 1)

Injected Contradiction Edges:

```
{
  "edges": [
    { "source": "belief_1", "target": "schema_1", "weight": 0.2, "type": "contradict" },
    { "source": "belief_2", "target": "belief_1", "weight": 0.2, "type": "contradict" }
  ]
}
```

}

- Two new contradiction edges destabilize both belief→schema and belief→belief.

Metrics After Paradox Injection

- **Total edge weight:** previous $1.15 + 0.2 + 0.2 = 1.55$
- **CD:** $(0.2+0.2) / (1.55+\epsilon) \approx 0.387$
- **Graph Entropy (H):** calculate using updated weights → $H \approx 1.743$
- **CM:** $1 / (1.743 + \epsilon) \approx 0.573$
- **PB:** unchanged from 0.348 (no new projects_to)
- **MaxD:** 0

Metric	Value
Contradiction Density	0.387
Coherence Mass	0.573
Projection Bias	0.348
Max Recursion Depth	0

Constraint Check:

- $CD(0.387) > \theta_c(0.35) \rightarrow$ violation
- CM, PB, MaxD within limits
→ **Rollback will be triggered** if automatic enforcement runs before reinforcement.

2. Schema Reinforcement (Cycle Step 2)

Assuming either enforcement is delayed for demonstration, we now reinforce:

```
{
  "edges": [
    { "source": "schema_1", "target": "belief_1", "weight": 0.15, "type": "reinforce" }
  ]
}
```

- Adds stabilizing reinforce edge from schema_1 back to belief_1.

Metrics After Reinforcement

- **Total edge weight:** $1.55 + 0.15 = 1.70$
- **CD:** remains $(0.4) / (1.70 + \epsilon) \approx 0.235$
- **Graph Entropy (H):** recalc $\rightarrow H \approx 1.650$
- **CM:** $1 / (1.650 + \epsilon) \approx 0.606$
- **PB:** unchanged ≈ 0.235
- **MaxD:** 0

Metric	Value
Contradiction Density	0.235
Coherence Mass	0.606
Projection Bias	0.235
Max Recursion Depth	0

Constraint Check: all metrics now within thresholds $\rightarrow$ stable recovery.

3. Observations and Audit

- **Destabilization:** Paradox injection raised CD above threshold, triggering enforcement.
- **Recovery:** Schema reinforcement drove CD well below threshold and boosted CM.

- **Audit Log:** Should contain entries for paradox mutation, rollback/skip, reinforcement, and metric deltas.
-

Next Steps

Thread #10 will explore recursive iterations of this cycle, examining how repeated destabilization and recovery affect long-term symbolic resilience metrics (e.g., resilience trends over multiple cycles).

Thread #10: Recursive Destabilization–Recovery Iterations & Resilience Analysis

In this thread, we simulate multiple cycles of paradox injection followed by schema reinforcement to evaluate the long-term symbolic resilience of the graph. We will outline pseudocode for the process, define resilience metrics, and describe how to analyze trends over successive cycles.

1. Objectives

- **Simulate** N repeated destabilization–recovery cycles on the same baseline graph.
 - **Record** key metrics (CD, CM, PB) after each destabilization and after each recovery.
 - **Compute** a resilience score per cycle: how quickly and how fully the system recovers into the safe threshold region.
 - **Analyze** trends to determine if resilience improves, degrades, or plateaus over time.
-

2. Pseudocode Outline

PARAMETERS

```

define N_cycles      # number of iterations
define paradox_weight # weight for each paradox injection
define reinforce_weight # weight for each reinforcement

# DATA STRUCTURES
initialize metrics_log = [] # list of (cycle, stage, CD, CM, PB)

# LOOP
for cycle in range(1, N_cycles + 1):
    # Stage 1: Paradox Injection
    inject_contradiction_edges(weight=paradox_weight)
    CD1, CM1, PB1 = recalc_metrics()
    if CD1 > 0c:
        rollback_to_last_stable_snapshot()
        CD1, CM1, PB1 = recalc_metrics()
    record(metrics_log, (cycle, 'post_paradox', CD1, CM1, PB1))

    # Stage 2: Schema Reinforcement
    inject_reinforce_edge(weight=reinforce_weight)
    CD2, CM2, PB2 = recalc_metrics()
    record(metrics_log, (cycle, 'post_reinforce', CD2, CM2, PB2))

# END LOOP

# ANALYSIS
# For each cycle:
# recovery_delta_CD = CD1 - CD2
# recovery_delta_CM = CM2 - CM1
# Compute resilience_score_cycle = (recovery_delta_CM) / recovery_time or similar
# Plot CD and CM trajectories over cycles to visualize resilience trends

```

3. Resilience Metrics Definitions

- **Recovery Delta CD:** $\Delta CD = CD_{\text{post_paradox}} - CD_{\text{post_reinforce}}$ \Delta CD = CD_{\text{\{post\}_paradox}} - CD_{\text{\{post\}_reinforce}}
- **Recovery Delta CM:** $\Delta CM = CM_{\text{post_reinforce}} - CM_{\text{post_paradox}}$ \Delta CM = CM_{\text{\{post\}_reinforce}} - CM_{\text{\{post\}_paradox}}
- **Resilience Score (per cycle):** could be defined as ratio of $\Delta CM / (CD_{\text{post_paradox}}) \Delta CM / (CD_{\text{\{post\}_paradox}})$ or simply $\Delta CM / \Delta CD$.

- **Long-Term Trend:** Analyze sequence of resilience scores across cycles 1..N to see stability.
-

4. Data Collection & Visualization

- **metrics_log** becomes a table with columns: `cycle`, `stage`, `CD`, `CM`, `PB`.
 - After simulation, export as CSV or JSON for plotting.
 - Visualize:
 - CD and CM vs cycle (two lines, `post_paradox` and `post_reinforce`).
 - Resilience Score vs cycle.
-

5. Next Steps

- **Thread #11** will formalize how to integrate these resilience analytics into the engine's reporting system, including pseudocode for generating summary reports and visualization hooks.
 - Optionally, adjust `paradoxweight``paradox_weight` and `reinforceweight``reinforce_weight` adaptively based on observed resilience to simulate dynamic adversarial vs defensive strategies.
-

End of Thread #10

Thread #11: Integrating Resilience Analytics into Reporting System

This thread details how to incorporate the resilience cycle analytics (from Thread #10) into the Janus engine's automated reporting framework. It outlines pseudocode for summary report generation and hooks for visualization.

1. Data Collection Hook

Within the engine's main loop or CLI command, after each destabilization–recovery cycle:

```
function run_resilience_cycles(N_cycles, paradox_weight, reinforce_weight):
    metrics_log = []
    for cycle in 1..N_cycles:
        // Paradox Injection and possible rollback
        inject_contradiction_edges(weight=paradox_weight)
        metrics1 = recalc_metrics()
        if metrics1.CD >  $\theta_c$ :
            rollback()
            metrics1 = recalc_metrics()
        metrics_log.append({cycle, 'paradox', metrics1})

        // Schema Reinforcement
        inject_reinforce_edge(weight=reinforce_weight)
        metrics2 = recalc_metrics()
        metrics_log.append({cycle, 'reinforce', metrics2})

    return metrics_log
```

This function returns a list of metric tuples for further analysis.

2. Report Generation Pseudocode

After `run_resilience_cycles`, generate a summary report:

```
function generate_resilience_report(metrics_log):
    report = {}
    report.total_cycles = max(metrics_log.cycle)

    // Compute per-cycle recovery deltas and resilience score
    for each cycle in 1..total_cycles:
        m_p = find metrics_log entry for (cycle, 'paradox')
```



```

m_r = find_metrics_log entry for (cycle, 'reinforce')
delta_CD = m_p.CD - m_r.CD
delta_CM = m_r.CM - m_p.CM
resilience_score = delta_CM // could refine
report.cycles[cycle] = {
    'pre_CD': m_p.CD, 'post_CD': m_r.CD,
    'pre_CM': m_p.CM, 'post_CM': m_r.CM,
    'delta_CD': delta_CD, 'delta_CM': delta_CM,
    'resilience_score': resilience_score
}

// Aggregate metrics
report.average_resilience = mean(report.cycles[*].resilience_score)
report.min_resilience = min(report.cycles[*].resilience_score)
report.max_resilience = max(report.cycles[*].resilience_score)

return report

```

The `report` structure can be serialized to JSON or printed as formatted text.

3. Visualization Hooks

Within a Jupyter notebook or CLI that supports ASCII/Unicode charts, provide helper functions:

```

function plot_metrics_over_cycles(report):
    cycles = list(report.cycles.keys())
    CD_paradox = [report.cycles[c].pre_CD for c in cycles]
    CD_reinforce = [report.cycles[c].post_CD for c in cycles]
    CM_paradox = [report.cycles[c].pre_CM for c in cycles]
    CM_reinforce = [report.cycles[c].post_CM for c in cycles]

    // Example: ASCII line chart or integrate with matplotlib if available
    plot_line(cycles, CD_paradox, label='CD after paradox')
    plot_line(cycles, CD_reinforce, label='CD after reinforcement')
    plot_line(cycles, CM_paradox, label='CM after paradox')
    plot_line(cycles, CM_reinforce, label='CM after reinforcement')

```

Alternatively, export `report` to CSV and instruct the user to load into spreadsheet or BI tool for visualization.

4. CLI Command Sketch

Introduce new CLI command in `cli.py`:

```
def resilience(
    n_cycles: int = typer.Option(5, '--cycles'),
    paradox_weight: float = typer.Option(0.2, '--paradox'),
    reinforce_weight: float = typer.Option(0.15, '--reinforce'),
    output: Optional[str] = typer.Option(None, '--out')
):
    metrics = run_resilience_cycles(n_cycles, paradox_weight, reinforce_weight)
    report = generate_resilience_report(metrics)
    if output:
        write_json(output, report)
        typer.echo(f'Resilience report saved to {output}')
    else:
        typer.echo(json.dumps(report, indent=2))
```

This provides end-to-end resilience analytics via the CLI.

Next Steps

- **Thread #12** will formalize how to extend the system to adaptive weight adjustment, where `paradox_weight` and `reinforce_weight` evolve based on observed resilience to create an adversarial training loop.
-

End of Thread #11

Thread #12: Adaptive Weight Adjustment for Adversarial Training

In this thread, we outline a mechanism to adaptively adjust paradox injection and schema reinforcement weights based on observed resilience scores from Thread #11. This creates an

adversarial training loop to systematically challenge and strengthen the symbolic graph's resilience.

1. Goals of Adaptive Adjustment

- **Targeted Stress Testing:** Increase paradox intensity when resilience is high, and reduce when low.
 - **Focused Reinforcement:** Boost reinforcement strength when recovery delta falls below a target.
 - **Dynamic Equilibrium:** Converge to an equilibrium where the graph maintains resilience within a narrow band under varying pressures.
-

2. Key Variables and Parameters

- `paradox_weight` (w_p): initial weight for contradiction edges.
 - `reinforce_weight` (w_r): initial weight for reinforcement edges.
 - `resilience_score_cycle`: as defined in Thread #10 (ΔCM per cycle).
 - `target_resilience` (R_t): desired minimum resilience threshold.
 - `adjustment_factor` (α): magnitude of weight change per cycle.
-

3. Adaptive Adjustment Algorithm (Pseudocode)

```
function adaptive_resilience_training(  
    N_cycles, w_p_init, w_r_init, R_t,  $\alpha$   
)  
    w_p = w_p_init  
    w_r = w_r_init  
    history = []
```

```

for cycle in 1..N_cycles:
    # Run one destabilization–recovery cycle using current weights
    metrics_log = run_resilience_cycles(1, w_p, w_r)
    cycle_data = extract_cycle_data(metrics_log, cycle)
    R = cycle_data.resilience_score

    # Record weights and resilience
    history.append({ 'cycle': cycle, 'w_p': w_p, 'w_r': w_r, 'resilience': R })

    # Adjust paradox weight: increase if resilience > target, else decrease
    if R > R_t:
        w_p = w_p * (1 +  $\alpha$ )
    else:
        w_p = w_p * (1 -  $\alpha$ )

    # Adjust reinforcement weight: increase if resilience < target, else decrease
    if R < R_t:
        w_r = w_r * (1 +  $\alpha$ )
    else:
        w_r = w_r * (1 -  $\alpha$ )

    # Clamp weights to [min_weight, max_weight]
    w_p = clamp(w_p, min=0.01, max=1.0)
    w_r = clamp(w_r, min=0.01, max=1.0)

return history

```

4. Monitoring and Visualization

- **History Table:** columns `cycle`, `w_p`, `w_r`, `resilience`.
 - **Plots:**
 - `w_p` and `w_r` vs. cycle to see weight trajectories.
 - `resilience` vs. cycle to see how adaptive process converges.
-

5. Integration into CLI

Extend the `resilience` command with an `--adaptive` flag and new options:

```
def resilience(
    n_cycles: int = 5,
    paradox_weight: float = 0.2,
    reinforce_weight: float = 0.15,
    adaptive: bool = False,
    target_resilience: float = 0.3,
    adjustment_factor: float = 0.1,
    output: Optional[str] = None
):
    if adaptive:
        history = adaptive_resilience_training(
            n_cycles, paradox_weight, reinforce_weight,
            target_resilience, adjustment_factor
        )
        serialize_or_print(history)
    else:
        # existing resilience flow
```

6. Next Steps

Thread #13 will document how to incorporate persistence and checkpointing in this adaptive loop, so intermediate states and weight values can be saved and resumed.

End of Thread #12

Thread #13: Persistence & Checkpointing for Adaptive Training

This thread designs how to persist intermediate state during adaptive resilience training (Thread #12), enabling pause/resume and fault tolerance.

1. Objectives

- **Checkpoint Weights & Metrics:** Save `paradox_weight`, `reinforce_weight`, and recent `resilience_score` after each cycle.
 - **Persist Graph Snapshot:** Store the current graph state so training can resume from the exact point of interruption.
 - **Resume Capability:** On restart, detect latest checkpoint and load weights, metrics history, and graph state to continue further cycles.
-

2. Directory & File Conventions

- **Checkpoint Folder:** `data/adaptive_checkpoints/`
- **Checkpoint File:** `checkpoint_cycle_<n>.json`

Each file contains a JSON object with:

```
{
  "cycle": n,
  "paradox_weight": w_p,
  "reinforce_weight": w_r,
  "metrics_log": [ /* list of previous cycle entries */ ],
  "snapshot_file": "data/snapshots/graph_<timestamp>.json"
}
```

3. Checkpoint Creation Pseudocode

```
function save_adaptive_checkpoint(
  cycle, w_p, w_r, metrics_log, engine
):
  snapshot = engine.snapshot_state()
  checkpoint = {
    "cycle": cycle,
    "paradox_weight": w_p,
    "reinforce_weight": w_r,
    "metrics_log": metrics_log,
    "snapshot_file": snapshot
```

```
}  
filepath = f"data/adaptive_checkpoints/checkpoint_cycle_{cycle}.json"  
write_json(filepath, checkpoint)  
return filepath
```

Call this at the end of each cycle in `adaptive_resilience_training`.

4. Resume Mechanism Pseudocode

```
function load_latest_checkpoint():  
    files = sorted(list_dir("data/adaptive_checkpoints"))  
    if not files:  
        return None  
    latest = files[-1]  
    return read_json(latest)  
  
function adaptive_resilience_training_with_resume(...):  
    ckpt = load_latest_checkpoint()  
    if ckpt:  
        start_cycle = ckpt.cycle + 1  
        w_p = ckpt.paradox_weight  
        w_r = ckpt.reinforce_weight  
        metrics_log = ckpt.metrics_log  
        engine.load_snapshot(ckpt.snapshot_file)  
    else:  
        start_cycle = 1  
        w_p, w_r = w_p_init, w_r_init  
        metrics_log = []  
  
    for cycle in start_cycle..N_cycles:  
        ... // same as Thread #12  
        save_adaptive_checkpoint(cycle, w_p, w_r, metrics_log, engine)
```

5. CLI Integration

Enhance `resilience --adaptive` command with a `--resume` flag:

```
janus resilience --adaptive --resume
```

- On `--resume`, load from latest checkpoint; otherwise start fresh.
-

6. Next Steps

Thread #14 will outline how to integrate real-time logging and user notifications (e.g., printing checkpoint info) and discuss cleanup policies for old checkpoints (e.g., keep last K files or compress snapshots).

End of Thread #13

Thread #14: Real-Time Logging, User Notifications & Checkpoint Cleanup Policies

In this thread, we enhance the adaptive training loop with live feedback and define policies for managing accumulated checkpoint files.

1. Real-Time Console Logging & Notifications

Goals

- Provide operators with immediate insights into each adaptive cycle.
- Report key data at the end of each cycle: cycle number, weights, resilience score.
- Optionally toggle verbosity levels (quiet, normal, verbose).

Pseudocode for Logging

```
function log_cycle_info(cycle, w_p, w_r, resilience, verbosity):  
    message = f"[Cycle {cycle}] paradox_weight={w_p:.3f}, reinforce_weight={w_r:.3f},  
    resilience={resilience:.3f}"
```



```
if verbosity == 'verbose':  
    message += " | detailed metrics: CD, CM, PB, MaxD ..."  
print(message)
```

Integration Points

- After computing `resilience_score` and before adjusting weights, call `log_cycle_info`.
 - Add `--verbosity` CLI flag with choices `[quiet, normal, verbose]` (default `normal`).
-

2. Checkpoint Cleanup Policies

Over time, adaptive checkpoints can consume storage. We propose:

A. Retention Strategy

- **Keep Last K Checkpoints:** Only the most recent `K` files are retained (e.g., `K=5`).
- **Purge Older Checkpoints:** Automatically delete older files beyond `K` after saving a new checkpoint.

B. File Compression

- **Compress Before Deletion:** Optionally gzip older checkpoints instead of outright deletion.

C. Manual Cleanup Command

Provide CLI command:

```
janus cleanup-checkpoints --keep 5 --compress
```

- - `--keep N` retains `N` most recent, deletes/compresses rest.

Pseudocode for Automatic Retention

```
function enforce_retention_policy(directory, keep):
    files = sorted(list_dir(directory), by=modification_time)
    old = files[0 : max(0, len(files) - keep)]
    for f in old:
        if compress_old:
            gzip_compress(f)
        else:
            delete_file(f)
```

Integrate this call at the end of each checkpoint save.

3. CLI Enhancements

New Flags

- `--verbosity [quiet,normal,verbose]` for `resilience --adaptive`.
- `cleanup-checkpoints` command with `--keep`, `--compress` options.

Example Usage

```
# Run adaptive training verbosely, then cleanup
janus resilience --adaptive --cycles 20 --verbosity verbose
janus cleanup-checkpoints --keep 5 --compress
```

4. Next Steps

Thread #15 will address integrating external visualization hooks (e.g., writing CSV or HTML dashboards), and discuss possible web UI extensions for live cycle monitoring.

End of Thread #14

Thread #15: External Visualization Hooks & Web UI Extension

This thread outlines how to export resilience and cycle metrics for external visualization (CSV, HTML dashboards) and proposes a simple web-based UI for live monitoring of adaptive training loops.

1. CSV Export Hook

Purpose

Enable analysts to load cycle data into spreadsheets or BI tools for custom charts.

Pseudocode

```
function export_metrics_csv(metrics_log, filepath):  
    // metrics_log: list of {cycle, stage, CD, CM, PB}  
    write_header("cycle,stage,CD,CM,PB")  
    for entry in metrics_log:  
        line = f"{entry.cycle},{entry.stage},{entry.CD:.4f},{entry.CM:.4f},{entry.PB:.4f}"  
        append_line(filepath, line)
```

Integration

- After running resilience cycles, call `export_metrics_csv(metrics_log, "resilience.csv")`.
 - Add `--csv-out <file>` CLI option to specify export path.
-

2. HTML Dashboard Generator

Purpose

Produce a self-contained HTML page with embedded interactive charts (e.g., using Chart.js) summarizing cycle metrics.

Pseudocode

```
function generate_html_dashboard(metrics_log, output_html):
    extract arrays for cycles, CD_paradox, CD_reinforce, CM_paradox, CM_reinforce
    html_template = load_template("dashboard_template.html")
    inject_json_data(html_template, {
        "cycles": cycles,
        "CD_paradox": CD_paradox,
        "CD_reinforce": CD_reinforce,
        "CM_paradox": CM_paradox,
        "CM_reinforce": CM_reinforce
    })
    write_file(output_html, html_template)
```

Template Outline (**dashboard_template.html**)

```
<!DOCTYPE html>
<html>
<head>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
    <canvas id="cdChart"></canvas>
    <canvas id="cmChart"></canvas>
    <script>
        const data = {{DATA_JSON}};
        new Chart(document.getElementById('cdChart'), {
            type: 'line', data: {
                labels: data.cycles,
                datasets: [
                    { label: 'CD after Paradox', data: data.CD_paradox },
                    { label: 'CD after Reinforce', data: data.CD_reinforce }
                ]
            }
        });
        // Similarly for CM chart
    </script>
</body>
</html>
```

3. Simple Flask Web UI

Purpose

Provide a lightweight local web server for real-time monitoring of current resilience training.

Pseudocode

```
from flask import Flask, render_template, jsonify
app = Flask(__name__)

state = { 'metrics_log': [], 'current_cycle': 0, 'w_p': 0, 'w_r': 0 }

@app.route('/')
def dashboard():
    return render_template('web_dashboard.html')

@app.route('/api/state')
def api_state():
    return jsonify(state)

# In the adaptive loop:
# after each cycle: state['metrics_log'].append(...); state['current_cycle'] = cycle; state['w_p'] = w_p; state['w_r'] = w_r
```

UI Template (**web_dashboard.html**)

- Use AJAX to poll **/api/state** every second.
- Update Chart.js charts and display numeric values and weight sliders.

4. Next Steps

Thread #16 will discuss packaging the web UI assets, security considerations (CORS, authentication), and deployment options (Docker + Gunicorn) for sharing dashboards within teams.

End of Thread #15

Thread #16: Web UI Packaging, Security, and Deployment

This thread covers how to package and serve the web dashboard safely, addressing asset bundling, CORS/authentication, and deployment (Docker + Gunicorn).

1. Packaging Web Assets

Directory Structure

```
/janus
  /web_ui
    /templates
      web_dashboard.html
    /static
      /js
        dashboard.js
      /css
        styles.css
```

- **templates/** holds Jinja2 HTML templates.
- **static/** holds JavaScript (Chart.js wrappers), CSS, and any images.

Build Process

- No build step for pure HTML/JS; if using modern JS frameworks, integrate a build (e.g., webpack).
 - Ensure versioned filenames (e.g., `dashboard.bundle.js?v=1.0.0`) for cache busting.
-

2. Security Considerations

CORS Configuration

If API and UI are served on separate origins, configure CORS headers in Flask:

```
from flask_cors import CORS
app = Flask(__name__)
CORS(app, resources={r"/api/*": {"origins": "https://your-domain.com"}})
```

-
- Restrict allowed origins to known client hosts.

Authentication & Authorization

- **Option A: HTTP Basic Auth** (simple credential check)
 - Use `flask-httpauth` to protect `/` and `/api/*` endpoints.
- **Option B: Token-Based Auth**
 - Implement API key or JWT issuance via `/login` endpoint.
 - Require `Authorization: Bearer <token>` header on API calls.

Transport Security

- Serve over HTTPS (TLS) in production.
- If behind a reverse proxy (NGINX), offload TLS there.

Rate Limiting

- Use `flask-limiter` to throttle API requests and prevent abuse.

3. Deployment Architecture

Docker + Gunicorn

Dockerfile for Web UI

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV FLASK_APP=janus_web:app
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "janus_web:app", "--workers", "4"]
```

- Installs dependencies, copies code, runs Gunicorn with 4 workers.

docker-compose.yml Example

```
version: '3.8'
services:
  janus_api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - FLASK_ENV=production
  nginx:
    image: nginx:stable-alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - janus_api
```

- **nginx.conf** proxies `/api/` to `janus_api:8000` and serves static files directly.

CI/CD Integration

- **GitHub Actions** can build Docker image, push to registry.
- On merge to `main`, trigger deployment to staging/production.

Monitoring & Logging

- Collect Gunicorn logs and Flask app logs via stdout.
 - Integrate with logging aggregator (e.g., ELK or Datadog).
 - Health-check endpoint `/healthz` returning 200 OK.
-

4. Environment & Configuration

- Store secrets (API keys, auth credentials) in environment variables or a secrets manager.
- Mount via Docker Compose:

environment:

- `AUTH_TOKEN=${AUTH_TOKEN}`
- `CORS_ALLOWED=${CORS_ALLOWED_ORIGINS}`

5. Next Steps

****Thread #17**** will propose scaling strategies (horizontal scaling, load balancing) and discuss high-availability patterns (multiple replicas, session persistence).

End of Thread #16

Thread #17: Scaling Strategies & High-Availability Patterns

This thread outlines how to scale the Janus web UI & API for high throughput, fault tolerance, and high availability.

1. Horizontal Scaling & Load Balancing

Approach

- **Multiple Instances:** Run several replicas of the Flask/Gunicorn service.
- **Load Balancer:** Place an NGINX or cloud LB (AWS ALB, GCP Cloud LB) in front to distribute traffic.
- **Session Affinity:** Not required if the API is stateless. If needed (e.g., WebSocket), enable sticky sessions.

Configuration Example (NGINX)

```
upstream janus_api {
    server janus_api_1:8000;
    server janus_api_2:8000;
    # health_check can be configured for upstream
}

server {
    listen 80;
    location /api/ {
        proxy_pass http://janus_api;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

2. Database & State Management

- **Stateless API:** The Flask service itself remains stateless. All state lives in:
 - **File storage:** snapshots, audit logs, checkpoints (use shared network storage like NFS or S3).

- **Optional DB:** For heavy production, store checkpoints and metrics in a database (PostgreSQL, Redis).
 - **Shared Storage Options:**
 - **Network File System (NFS):** Mount `data/` directory across replicas.
 - **S3 or Blob Storage:** Upload snapshots and checkpoints to cloud buckets; load them on demand.
-

3. Caching & Performance Optimization

- **In-Memory Cache:** Use Redis or Memcached for caching frequently accessed metrics or snapshots.
 - **CDN for Static Assets:** Serve `/static/` files (JS/CSS) via a CDN for low-latency global access.
-

4. High-Availability & Failover

- **Multiple Data Centers:** Deploy instances in multiple availability zones.
 - **Health Checks & Auto-Recovery:** Configure cloud auto-scaling groups to replace failed instances.
 - **Database HA:** If using a database, enable master-slave or multi-master clustering.
-

5. Monitoring & Alerting

- **Health Endpoint:** `/healthz` returns 200 if all subsystems (file storage, DB, cache) are reachable.

- **Metrics Export:** Integrate Prometheus to collect metrics (request latency, error rates, CPU/memory).
 - **Alerting:** Set alerts for high 5xx rates, high latency, low resilience scores trending.
-

6. Disaster Recovery

- **Backups:** Regular backups of file storage or database snapshots.
 - **Restore Procedures:** Document how to restore snapshots, roll back to previous code versions, and replay audit logs.
-

7. Next Steps

Thread #18 will discuss advanced security — rate limiting strategies, WAF integration, and DDoS mitigation for the Janus API.

End of Thread #17

Thread #18: Advanced Security — Rate Limiting, WAF Integration, and DDoS Mitigation

This thread discusses how to fortify the Janus API and Web UI against abusive traffic, ensuring reliability under attack and compliance with best practices.

1. Rate Limiting

Purpose

Prevent excessive requests from a single client or IP, protecting backend resources and maintaining fair usage.

Implementation with `flask-limiter`

```
from flask import Flask
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

app = Flask(__name__)
# Global rate limit: 100 requests per minute per IP
limiter = Limiter(app, key_func=get_remote_address, default_limits=["100 per minute"])

@app.route('/api/compute_metrics')
@limiter.limit("10 per minute") # stricter limit on heavy endpoint
def compute_metrics():
    ...
```

Considerations

- Apply stricter limits on mutation endpoints (`inject_paradox`, `run_plugin`).
 - Provide informative `Retry-After` headers.
 - Integrate with Redis backend for distributed limits when running multiple instances.
-

2. Web Application Firewall (WAF) Integration

Purpose

Block malicious patterns (SQL injection, cross-site scripting, known bot traffic) at the network edge.

Options

- **Cloud WAFs:** AWS WAF, Cloudflare WAF, Azure Front Door WAF.

- **Self-Managed:** ModSecurity with NGINX or Apache.

Example: AWS WAF

1. **Define rules** for IP blacklisting, rate-based rules, and OWASP Core Rule Set.
 2. **Associate** the WAF with your Application Load Balancer or API Gateway.
 3. **Logging & Monitoring:** Enable real-time metrics and CloudWatch alarms on blocked requests.
-

3. DDoS Protection

Purpose

Absorb and mitigate large-scale volumetric attacks aimed at overwhelming network bandwidth or server capacity.

Strategies

- **CDN Shield:** Use Cloudflare, AWS CloudFront, or Fastly to cache static assets and terminate edge TLS.
- **Anycast Networks:** Distribute traffic globally to prevent saturation of a single link.
- **Scrubbing Services:** Route suspicious traffic through DDoS scrubbing centers (e.g., Akamai Kona, Arbor).

Best Practices

- Keep critical endpoints behind the CDN/WAF layer.
 - Use SYN cookies and TCP rate limiting at load balancer.
 - Monitor traffic patterns; set thresholds for alerting unusual spikes.
-

4. Security Monitoring & Incident Response

- **SIEM Integration:** Forward logs (Gunicorn, Flask, WAF) to a SIEM (Splunk, ELK, Datadog).
 - **Alerting:** Configure alerts for repeated 429 (rate limit) or 403 (WAF block) responses.
 - **Playbooks:** Document steps to identify and block new attack vectors; maintain an incident response runbook.
-

5. Next Steps

Thread #19 will outline comprehensive performance testing and benchmarking strategies, including load tests (Locust, JMeter) and chaos engineering experiments (Simian Army style) to validate both resilience and security under real-world conditions.

End of Thread #18

Thread #19: Performance Testing & Benchmarking

This thread outlines comprehensive strategies for performance testing, load benchmarking, and chaos engineering to validate system resilience under realistic and adverse conditions.

1. Load Testing Tools & Scenarios

1.1 Locust

- **Nature:** Python-based, user-centric load testing.
- **Setup:** Define user behaviors in `locustfile.py`.

- **Example Scenario:** Simulate 100 concurrent users calling:

- `/api/compute_metrics`
- `/api/run_plugin/schema_mapper`
- `/api/inject_paradox`

Pseudocode (locustfile.py)

```
from locust import HttpUser, task, between
```

```
class JanusUser(HttpUser):
```

```
    wait_time = between(1, 3)
```

```
    @task(3)
```

```
    def compute_metrics(self):
```

```
        self.client.get("/api/compute_metrics")
```

```
    @task(1)
```

```
    def inject_paradox(self):
```

```
        self.client.post(
            "/api/inject_paradox", json={"intensity": 0.2}
        )
```

1.2 JMeter

- **Nature:** Java-based, GUI or CLI mode, rich assertion options.
- **Setup:** Create a Test Plan with Thread Groups, HTTP Requests, and Listeners.
- **Assertions:** Check response codes, latency thresholds (<200ms).

1.3 k6 (Optional)

- **Nature:** JavaScript-based, lightweight CLI.
- **Script Example:**

```
import http from 'k6/http';
import { check, sleep } from 'k6';
```



```
export let options = { vus: 50, duration: '2m' };

export default function () {
  let res = http.get('https://api.example.com/compute_metrics');
  check(res, { 'status was 200': (r) => r.status === 200 });
  sleep(1);
}
```

2. Benchmarking Metrics

- **Latency Percentiles:** p50, p90, p99 for each endpoint.
- **Throughput (RPS):** Requests per second handled.
- **Error Rate:** Percentage of 5xx or failed requests.
- **Resource Utilization:** CPU, memory, and I/O under load.

Data Collection

- Push metrics to Prometheus/Grafana during tests.
 - Use JMeter Listeners or Locust integrations for real-time charts.
-

3. Chaos Engineering Experiments

3.1 Simian Army Style

- **Tools:** Chaos Monkey, Gremlin, or custom scripts.
- **Target Faults:**
 - **Instance Termination:** Randomly kill one application instance.
 - **Network Latency:** Introduce artificial delays (tc/netem).

- **Disk I/O Errors:** Simulate I/O faults on snapshot storage.

3.2 Experiment Workflow

1. **Baseline:** Run load test at steady state.
 2. **Inject Fault:** E.g., kill a container or inject latency.
 3. **Observe:** Check SLA metrics (latency, errors) remain within thresholds.
 4. **Recover:** Verify auto-scaling or instance restart restores capacity.
-

4. Test Automation & CI Integration

- **GitHub Actions** or Jenkins pipelines to:
 - Spin up test environment.
 - Run Locust or JMeter scripts.
 - Fail builds if error rate >1% or p99 latency >500ms.
 - Archive test reports (HTML, CSV).
 - **Thresholds as Code:** Define acceptable performance thresholds in code/config for automated gating.
-

5. Next Steps

Thread #20 will propose best practices for regression testing of cognitive metrics logic, ensuring that changes to the engine don't unintentionally alter metric definitions (CD, CM, PB).

End of Thread #19

Thread #20: Regression Testing for Cognitive Metrics Logic

This thread defines best practices and strategies to ensure that updates to the engine do not unintentionally alter the definitions or computations of key metrics: Contradiction Density (CD), Coherence Mass (CM), Projection Bias (PB), and Max Recursion Depth.

1. Golden Master Tests

Purpose

Capture a known-good snapshot of metric outputs for a fixed graph and compare against future runs to detect unintended changes.

Steps

1. **Select Representative Graphs:** Create several JSON snapshots covering common and edge-case topologies (empty graph, linear chain, balanced tree, echo chamber).
2. **Record Baseline Metrics:** For each graph, record the tuple (CD, CM, PB, MaxDepth) into a `golden_metrics.json` file.
3. **Test Harness:** Implement a pytest function that loads each snapshot, computes metrics, and asserts equality (or near-equality within tolerance) to the golden values.

Pseudocode (pytest)

```
import json
import pytest
from janus_core.engine import JanusGraphEngine

with open('tests/golden_metrics.json') as f:
    golden = json.load(f)

@pytest.mark.parametrize('case', golden.keys())
def test_golden_metrics(case):
    path, expected = golden[case]['file'], golden[case]['metrics']
    engine = JanusGraphEngine()
    engine.load_snapshot(path)
```

```
actual = engine._current_metrics()
for key in expected:
    assert actual[key] == pytest.approx(expected[key], rel=1e-6)
```

2. Property-Based Tests (Hypothesis)

Purpose

Validate general mathematical properties of metrics across a wide range of generated graphs.

Examples

- **Bounds Testing:** $CD, CM, PB \in [0, 1]$ (or ≥ 0 for CM).
- **Monotonicity Tests:** Adding a contradiction edge should not decrease CD.
- **Symmetry Tests:** Projection bias computed on reversed graph yields same PB.

Pseudocode (Hypothesis)

```
from hypothesis import given, strategies as st
import networkx as nx
from janus_core.engine import JanusGraphEngine

@st.composite
def random_graph(draw):
    nodes = draw(st.integers(min_value=0, max_value=10))
    g = nx.gnp_random_graph(nodes, 0.3, directed=True)
    return g

@given(g=random_graph())
def test_cd_bounds(g):
    engine = JanusGraphEngine()
    engine.G = g
    cd = engine.contradiction_density()
    assert 0 <= cd <= 1
```

3. Incremental Regression Checks in CI

Purpose

Automatically run regression tests on every code change to catch metric deviations early.

Integration

- **GitHub Actions:** After unit tests, run `pytest tests/test_golden_metrics.py, pytest tests/test_properties.py`.
 - **Threshold Alerts:** If any metric assertion fails, block merge and report the diff.
-

4. Tolerance Management

Floating-Point Variability

- Allow small relative or absolute tolerances (e.g., `eps=1e-6`) in assertions.
- Document tolerance levels in `pytest.ini` or test code comments.

Configurable Tolerances

- Expose tolerance settings via `janus_config.json` section for tests.
-

5. Change Approval Workflow

1. **Developer Run:** Developer updates engine, runs full regression suite locally.
 2. **CI Validation:** CI runs regression; if metrics changed, tests fail.
 3. **Review:** Developer inspects metric diffs, confirms intentional or corrects errors.
 4. **Approve:** Merge only after regression suite passes.
-

6. Next Steps

Thread #21 will explore end-to-end integration testing with sample user workflows, simulating CLI interactions and plugin executions to validate complete system behavior.

End of Thread #20

Thread #21: End-to-End Integration Testing & CLI Workflow Validation

This thread defines strategies for validating the complete Janus system through scripted end-to-end tests, simulating real user workflows via the CLI and plugin executions.

1. Goals of Integration Testing

- **Verify** that CLI commands correctly orchestrate the engine, plugins, and persistence layers.
 - **Ensure** that typical user workflows (load, mutate, compute metrics, snapshot, audit) succeed without errors.
 - **Validate** that plugin mutations are honored and that rollback logic triggers appropriately when combined actions breach thresholds.
-

2. Test Harness Design

2.1 Use a Shell Script or Python Wrapper

- **Approach A:** Bash script (`tests/integration.sh`) invoking the `janus` CLI with assertions on exit codes and output patterns.

- **Approach B:** Python script using `subprocess` to call CLI commands and capture stdout/stderr for assertions.

2.2 Shared Test Workspace

- Create a temporary directory for each test run (`tmp/integration_test_{n}`).
 - Copy sample graph snapshots into it.
 - Clean up after test.
-

3. Sample End-to-End Scenarios

3.1 Workflow: Load, Inject Paradox, Compute Metrics, Save

1. **Load** a known small graph: `janus load sample_small.json`
2. **Inject Paradox:** `janus inject-paradox 0.5`
 - Expect: Non-zero contradiction density printed, or rollback error if threshold exceeded.
3. **Compute Metrics:** `janus compute-metrics`
 - Capture and parse output: verify format `CD: x.xxx, CM: x.xxx, PB: x.xxx, MaxDepth: N`
4. **Save:** `janus save modified.json`
 - Verify `modified.json` exists and contains updated graph state.

3.2 Workflow: Plugin Execution & Rollback

1. **Load** base graph.

2. **Run Plugin:** `janus run-plugin bias_scanner "confirmation bias detected"`
 - Plugin should add contradiction nodes/edges.
 3. **Compute Metrics:** ensure CD increased from baseline.
 4. **Run Plugin** again with payload to exceed paradox tolerance.
 - Expect: CLI prints rollback message and process exits with non-zero code.
 5. **Audit Log:** `janus audit-log`
 - Verify entries include both successful mutations and a `safety_violation` event.
-

4. Assertions & Validation

- **Exit Codes:** 0 for success, non-zero for safety rollback or errors.
 - **Output Patterns:** Regular expressions to match metric lines and plugin result JSON.
 - **File Existence:** Snapshot and output JSON files created/updated as expected.
 - **Audit Log Contents:** Must contain sequences of `mutation` and `safety_violation` events.
-

5. CI Integration

Add a job step in CI pipeline after unit tests:

```
- name: Run integration tests  
run: bash tests/integration.sh
```

-
- Ensure that failures in any step cause CI to fail and report which scenario failed.

6. Next Steps

Thread #22 will outline documentation templates for developer guides and user onboarding, ensuring that new users can follow these CLI workflows without confusion.

End of Thread #21

Thread #22: Documentation Templates for Developer Guides & User Onboarding

This thread provides skeleton templates and outlines for the key documentation artifacts your project needs, ensuring clear onboarding for both developers and end-users.

1. README.md (Project Overview)

Sections:

1. **Project Title & Tagline**
2. **Introduction**
 - Brief description of the Janus platform—a symbolic cognitive recursion engine.
3. **Key Features**
 - List core capabilities: graph engine, metrics (CD/CM/PB), rollback, plugins, CLI.
4. **Getting Started**
 - Prerequisites: Python 3.12+, Git, Docker (optional).
 - Installation (pip or Docker).
5. **Usage Examples**

- CLI commands: load, inject-paradox, run-plugin, compute-metrics, audit-log.

6. Documentation Links

- Developer Guide, Plugin Guide, API Reference, Ethical Spec, Math Structures.

7. Contributing

- Code style, testing, pull request process.

8. License & Code of Conduct

2. Developer Guide (docs/Developer_Guide.md)

Sections:

1. Project Architecture

- Directory layout
- Module responsibilities (janus_core, janus_modules, cli, tests)

2. Configuration

- janus_config.json schema
- Environment variables overrides

3. Data Models

- Description of NodeData & EdgeData fields

4. Core Engine

- How JanusGraphEngine works (transactions, metrics, rollback)
- Audit log format & hash chain

5. Writing Plugins

- Plugin interface `process(engine, payload)`
- Example template and best practices

6. Testing

- Unit tests (metrics, rollback)
- Integration tests (scripts, CI configuration)

7. Building & Packaging

- PyInstaller spec, Dockerfile, release process

8. Troubleshooting

- Common errors and resolutions

3. User Onboarding / Quick Start (docs/Quick_Start.md)

Sections:

1. Installation

- `pip install janus-cognitive`
- Docker usage

2. First Steps

- Create a new graph: `janus load --help`
- Inject paradox: `janus inject-paradox 0.3`
- Run schema mapping: `janus run-plugin schema_mapper "my text"`

3. Reading Metrics

- Interpret CD, CM, PB, MaxDepth outputs

4. Saving & Loading

- `janus save graph1.json / janus load graph1.json`

5. Viewing Audit Logs

- `janus audit-log | less`

6. Next Steps

- Explore plugins, write your own, check docs

4. CLI Reference (docs/CLI_Reference.md)

Sections:

1. Global Options

- `--help, --config, --strict`

2. Commands

- `load <file>`
- `save <file>`
- `inject-paradox <intensity>`
- `run-plugin <name> <payload>`
- `compute-metrics`
- `audit-log`

3. Examples

- Combined workflows
-

5. FAQ & Troubleshooting (docs/FAQ.md)

Sections:

1. General

- What is Janus?
- How is it different from other graph tools?

2. Metrics

- Why did my CD suddenly drop to zero?
- How to adjust thresholds in janus_config.json

3. Errors

- Ethical rollback triggered
- Audit log not found

4. Extensibility

- How to add a new plugin
-

End of Thread #22

Thread #23: API Reference Documentation

This thread defines the HTTP/REST API endpoints exposed by Janus, including request parameters, response schemas, and example payloads.

1. Authentication & Headers

Authentication: (Optional) If enabled, use API key via header:

Authorization: Bearer <API_KEY>

- - **Common Headers:**
 - `Content-Type: application/json`
 - `Accept: application/json`
-

2. Endpoints

2.1 GET `/api/compute_metrics`

- **Description:** Compute and return current graph metrics.
- **Query Parameters:**
 - None

Response (200 OK):

```
{
  "CD": 0.125,      // Contradiction Density
  "CM": 4.562,      // Coherence Mass
  "PB": 0.083,      // Projection Bias
  "MaxDepth": 3     // Maximum recursion depth
}
```

- - **Errors:**
 - `500 Internal Server Error` on unexpected failures.
-

2.2 POST `/api/inject_paradox`

- **Description:** Inject a batch of paradox (contradiction) edges.

Request Body:

```
{
  "edges": [           // Optional alternative to intensity
    {"source": "A", "target": "B", "weight": 0.3},
    {"source": "C", "target": "D", "weight": 0.5}
  ],
  "intensity": 0.2      // Fraction of existing nodes to link
}
```

-

Response (200 OK):

```
{
  "status": "ok",
  "injected": 5,          // Number of edges added
  "metrics": {           // Post-injection metrics
    "CD": 0.220,
    "CM": 4.120,
    "PB": 0.083,
    "MaxDepth": 4
  }
}
```

-

- **Errors:**

- **400 Bad Request** for malformed JSON or missing fields.

409 Conflict if rollback triggered due to thresholds exceeded:

```
{
  "status": "error",
  "error": "Ethical rollback triggered: contradiction_density exceeded"
}
```

-

2.3 POST `/api/run_plugin/{plugin_name}`

- **Description:** Execute a registered plugin with a string payload.
- **Path Parameter:**
 - `plugin_name` (string) — e.g. `schema_mapper`, `bias_scanner`, `persona_simulator`

Request Body:

```
{  
  "payload": "your input text or data here"  
}
```

-

Response (200 OK):

```
{  
  "status": "ok",  
  "plugin": "schema_mapper",  
  "output": {  
    "graph_delta": "added schema chain"  
  },  
  "metrics": { "CD": 0.125, "CM": 4.562, "PB": 0.083, "MaxDepth": 3 }  
}
```

-

- **Errors:**
 - `404 Not Found` if plugin name is unknown.
 - `400 Bad Request` for missing payload.
 - `409 Conflict` on rollback during plugin execution.

2.4 GET `/api/audit_log`

- **Description:** Retrieve the append-only audit log entries.
- **Query Parameters:**
 - `limit` (integer, optional) — number of most recent entries.

Response (200 OK):

```
[
  {"timestamp":"2025-07-12T14:00:00Z","event":"mutation",...},
  {"timestamp":"2025-07-12T14:01:12Z","event":"safety_violation",...}
]
```

-
- **Errors:**
 - `404 Not Found` if audit log file is missing.

3. Error Response Schema

All error responses use the following JSON schema:

```
{
  "status": "error",
  "error": "Detailed error message here"
}
```

4. Versioning & Deprecation

- **Base Path:** `/api/v1/` prefix (future versions).

Deprecation Policy: Mark endpoints as deprecated in headers:

Deprecation: true
 Sunset: Wed, 12 Aug 2025 00:00:00 GMT

-

5. Next Steps

Thread #24 will propose best practices for monitoring API usage metrics, SLIs/SLOs, and alerting on anomalies.

End of Thread #23

Thread #24: Monitoring, SLIs/SLOs, and Alerting

This thread outlines best practices for monitoring Janus system health, defining Service Level Indicators (SLIs) and Service Level Objectives (SLOs), and setting up alerting on anomalies.

1. Observability Stack

- **Metrics:** Collected via Prometheus exporters embedded in the Janus API and CLI (e.g., `prometheus_client`).
 - **Dashboards:** Visualized in Grafana or another dashboard tool.
 - **Logging:** Structured JSON logs sent to ELK/EFK or Datadog for search and correlation.
 - **Tracing (optional):** Distributed traces with OpenTelemetry for expensive operations.
-

2. Key Service Level Indicators (SLIs)

2.1 Availability

- **Endpoint Success Rate:** Percentage of HTTP `200` responses vs total requests for critical APIs (`compute_metrics`, `inject_paradox`, `run_plugin`).

2.2 Latency

- **p50, p90, p99 Latency:** Measured in milliseconds per endpoint.

2.3 Error Rate

- **5xx and Rollback Rate:** Fraction of requests resulting in server errors or ethical rollbacks (**409 Conflict**).

2.4 Throughput

- **Requests Per Second (RPS):** Overall traffic and per-endpoint.

3. Defining Service Level Objectives (SLOs)

SLI	SLO
Endpoint Success Rate	≥ 99.9% success over rolling 30 days
p90 Latency for <code>/compute_metrics</code>	< 200 ms over rolling 7 days
Error Rate (5xx & rollbacks)	< 0.1% over rolling 7 days
Throughput	≥ 50 RPS sustained under normal load

4. Error Budget and Burn Rate

1. **Error Budget:** Allowable failure margin (1 - SLO).
2. **Burn Rate:** Rate at which the budget is consumed.
3. **Alerts:**
 - **Burn Rate Alert:** Trigger if budget consumption > 2× the allowable rate.

- **Budget Exhaustion:** Alert when remaining budget < 20%.
-

5. Alerting Strategy

- **Critical Alerts** (PagerDuty):
 - SLO breaches (error budget exhaustion).
 - High 5xx rate or rollback rate spike.
 - **Warning Alerts** (Slack/email):
 - Latency p99 > threshold.
 - Throughput drop under normal load.
 - **Dashboard Annotations:** Mark deployments and incidents on time-series charts.
-

6. Next Steps

Thread #25 will outline incident response and postmortem procedures to ensure rapid recovery and continuous improvement.

End of Thread #24

Thread #25: Incident Response & Postmortem Procedures

This thread defines structured processes for responding to incidents affecting the Janus platform, and carrying out thorough postmortems to drive continuous improvement.

1. Incident Response Phases

1.1 Preparation

- **Runbooks & Playbooks:** Maintain up-to-date runbooks for common incident types (security breach, service outage, performance degradation).
- **Communication Plan:** Define stakeholder roles (Incident Commander, Scribe, Liaison), notification channels (PagerDuty, Slack), and escalation policies.
- **Drills:** Conduct regular tabletop exercises and game days to practice response.

1.2 Identification

- **Detection:** Use monitoring alerts (SLI breaches, error spikes) to detect anomalies.
- **Triage:** Assess severity based on impact to SLOs, customer impact, and business criticality.
- **Classification:** Tag incident type (e.g., security, performance, data integrity).

1.3 Containment

- **Short-term Mitigation:** Apply rate-limits, disable runaway plugins, scale up resources, or roll back recent deployments.
- **Isolation:** Quarantine affected services (e.g., isolate a bad node or container) to prevent propagation.
- **Communication:** Notify stakeholders of status, scope, and immediate mitigation steps.

1.4 Eradication

- **Root Cause Analysis (RCA):** Use logs, traces, and audit records to pinpoint the underlying fault.
- **Remediation Steps:** Patch code, update configurations, fix misbehaving modules, revoke compromised credentials.

- **Verification:** Test fixes in staging, then gradually roll back changes into production under careful observation.

1.5 Recovery

- **Restore Services:** Bring systems back to normal operation.
- **Monitor Closely:** Increase sampling frequency of SLIs, watch for relapse of symptoms.
- **Stakeholder Update:** Communicate resolution and time to recovery.

1.6 Lessons Learned

- **Postmortem Document:** Capture timeline, impact, root cause, remediation, and action items.
 - **Blameless Culture:** Focus on system improvements, not individual fault.
 - **Action Item Tracking:** Assign owners, implement changes (e.g., additional tests, improved runbooks, code refactors).
-

2. Postmortem Structure

1. **Summary:** Brief description of the incident and its impact.
2. **Timeline:** Detailed timeline of events (timestamps in UTC).
3. **Detection & Response:** How was the incident detected and triaged?
4. **Root Cause:** Technical explanation, supported by logs/audit entries.
5. **Resolution & Recovery:** Steps taken to fix and verify.
6. **Preventive Measures:** Code changes, monitoring improvements, runbook updates.
7. **Action Items:** List of tasks with owners and due dates.
8. **Follow-up:** Schedule follow-up meetings to review progress.

3. Tooling & Automations

- **Incident Management Platform:** Use PagerDuty or Opsgenie to coordinate alerts and on-call rotations.
- **Postmortem Templates:** Store in /docs/Postmortem_Template.md, linking to automated metrics & logs.
- **Audit Integrations:** Leverage the Janus audit log to reconstruct state before and during the incident.

4. Continuous Improvement

- **Retro Reviews:** Conduct a formal retro within 1 week of the incident.
- **Metric Calibration:** Adjust SLO thresholds or alert sensitivities based on learnings.
- **Runbook Updates:** Incorporate lessons into runbooks and playbooks.
- **Training:** Provide training sessions or documentation updates to the team.

5. Next Steps

Thread #26 will define a risk registry and mitigation plan, cataloging top project risks with likelihood, impact, and countermeasures.

End of Thread #25

Thread #26: Risk Registry & Mitigation Plan

This thread catalogs the top project risks for the Janus platform—covering technical, operational, ethical, and market dimensions—along with likelihood, impact, and proposed mitigations.

#	Risk Description	Likelihood	Impact	Mitigation Strategies
1	Recursion & Paradox Bugs Complex recursive operators may introduce non-obvious failures or infinite loops.	High	Critical	- Extensive unit & property tests- Fuzz testing with random graphs- Safety rollback enforced on recursion depth breaches.
2	Performance at Scale Graph metrics (entropy, depth) on large graphs (>10k nodes) may become prohibitively slow.	Medium	High	- Benchmark & profile critical paths- Implement sampling or incremental updates- Offload heavy calculations asynchronously.
3	Ethical Misuse Risk of deploying paradox injection or bias tools in unethical scenarios or without consent.	Medium	Severe	- Embed strict consent & rollback parameters in CLI/API- Audit log with hash chaining- Separate simulator vs deployment modes with explicit warnings.
4	Plugin Vulnerabilities Third-party or custom plugins could crash the engine or introduce security holes.	Medium	High	- Sandbox plugin execution where possible- Validate and review plugin code- Provide a plugin security checklist and signing mechanism.
5	Market Adoption Challenges Abstract nature of metrics (CD, CM, PB) may deter non-technical users.	High	Medium	- Develop clear user onboarding docs and examples- Provide default templates & presets- Offer training sessions and interactive tutorials.
6	Infrastructure Security Exposed API endpoints may be targeted by DDoS, injection attacks, or abuse.	Medium	High	- Implement rate limiting and WAF integration- Use CDN & DDoS protection services- Monitor 429/403 spikes and automate blacklisting.

7	Data Integrity & Logging Overload Audit logs and snapshots may grow unbounded, consuming storage.	Medium	Medium	- Implement log rotation and archival policies- Summarize older audit entries- Enforce retention policies via automated cleanup.
8	Compliance & Regulatory Risk Potential issues when used in clinical or defense contexts requiring certifications.	Low	High	- Engage legal/compliance early- Provide transparent ethical spec and audit trails- Offer on-premise deployments for regulated environments.

Next Steps

Thread #27 will propose a detailed data retention and archival policy, ensuring audit logs and snapshots are managed sustainably.

End of Thread #26

Thread #27: Data Retention & Archival Policy

This thread defines policies and procedures for retaining, rotating, and archiving data generated by the Janus platform—specifically audit logs, graph snapshots, and alignment records—ensuring compliance, cost control, and forensic readiness.

1. Data Categories & Retention Requirements

Data Type	Description	Retention Period	Storage Tier
Audit Logs	Append-only JSONL with mutation & rollback events	90 days	Hot (local SSD)
Graph Snapshots	Full JSON snapshots of graph state	30 days	Hot (local SSD)

Alignment Store	Side-channel JSONL of DOG/GOD alignment changes	30 days	Hot (local SSD)
Archived Snapshots	Older snapshots beyond 30 days	1 year	Cold (object store)
Archived Audit Logs	Audit logs older than 90 days	1 year	Cold (object store)

2. Rotation & Deletion Policies

1. **Daily Cleanup Job** (cron or scheduled task):

- Delete local snapshots older than 30 days.
- Delete audit logs older than 90 days from local storage (but keep a pointer to archived copies).

2. **Monthly Archival Job**:

- Compress and upload snapshots older than 30 days to long-term object storage (e.g., S3, Azure Blob) under `archive/snapshots/YYYY/MM/`.
- Compress and upload audit logs older than 90 days under `archive/audit_logs/YYYY/MM/`.

3. **Retention Enforcement**:

- After successful archival and verification, delete the compressed files older than the 1-year retention threshold from object storage to control costs.
-

3. Archival Format & Integrity

- **Compression:** Use gzip or zstd for JSONL and snapshot archives.
- **File Naming Convention:**

- Snapshots: `snapshot_YYYYMMDD_HHMMSS.json.gz`
 - Audit Logs: `audit_YYYYMMDD.jsonl.gz`
 - **Integrity Checks:**
 - Compute and store SHA256 checksums alongside each archive.
 - Verify checksum before deletion from primary storage.
-

4. Automation & Monitoring

- **Automation Scripts:** Implement as Python scripts or shell jobs, scheduled via cron, Airflow, or Kubernetes CronJob.
 - **Monitoring:** Track archival job success/failure in logs and emit metrics (e.g., `janus_archive_success_total`, `janus_archive_failure_total`).
 - **Alerting:** Alert if archival or cleanup jobs fail more than 3 consecutive times.
-

5. Compliance & Audit Readiness

- **Audit Trail:** Maintain an index of archived files with metadata (date range, checksum, storage location).
 - **Access Controls:** Restrict deletion and archive operations to authorized service accounts.
 - **Documentation:** Publish this policy in `docs/Data_Retention_Policy.md` and include examples of how to perform manual restores.
-

6. Next Steps

Thread #28 will propose a comprehensive plugin marketplace strategy—including governance, packaging guidelines, and discovery protocols—to foster community-driven extensions safely.

End of Thread #27

Thread #28: Plugin Marketplace Strategy

This thread outlines the design and governance of a Janus plugin marketplace, enabling safe community-driven extension of the core system while maintaining ethical, security, and quality standards.

1. Objectives

1. **Discoverability:** Provide a central registry for plugins that users can browse and install.
 2. **Safety & Trust:** Ensure all plugins meet minimum security and ethical guidelines.
 3. **Governance:** Define submission, review, and deprecation processes.
 4. **Compatibility:** Guarantee plugins work with core Janus versions via semantic versioning.
 5. **Monetization (optional):** Allow premium or certified plugins in a separate tier.
-

2. Marketplace Architecture

flowchart LR

```
subgraph Core Janus Platform
    A[janus CLI/API] --> B[Plugin Manager]
end
subgraph Marketplace Service
    C[Registry API]
    D[Web UI]
    E[Storage: pkg repos]
end
B -->|fetch plugin list| C
```

B -->|download plugin zip| E
D --> C
C --> E

- **Registry API:** Exposes endpoints for listing, searching, and retrieving plugin metadata (name, version, author).
- **Storage:** Hosts packaged plugin artifacts (tar.gz or wheel).
- **Web UI/CLI:** Allows users to search and install plugins via `janus plugin install <name>`.

3. Packaging Guidelines

Structure: Each plugin is a Python package with:

```
plugin_name/  
setup.py or pyproject.toml  
plugin_name/  
  __init__.py  
  process.py    # must expose process(engine, payload)
```

-
- **Metadata:** Must declare in `setup.py` or `pyproject.toml`:
 - `name`: `janus-plugin-<identifier>`
 - `version`: Semantic versioning (MAJOR.MINOR.PATCH)
 - `entry_point`: under `janus.plugins`, pointing to `module:process`
- **Dependencies:** Limit to necessary packages; avoid conflicting versions.
- **Signing (optional):** Plugins can include a GPG signature for publisher verification.

4. Submission & Review Process

1. **Submission Portal:** Authors submit plugin package URLs and metadata via web form or CLI.
 2. **Automated Checks:** Registry runs CI to:
 - Validate entry points and metadata.
 - Run smoke tests: import plugin, execute a simple `process(engine, {})`, ensure no exceptions.
 - Static analysis: linting, security scanners (bandit).
 3. **Manual Review:** Core team reviews for:
 - Ethical compliance (no hidden rollback bypasses).
 - Security (no exec of arbitrary code, sandboxing guidelines).
 4. **Approval & Publication:** Upon passing, plugin metadata and artifact are published.
 5. **Deprecation:** Plugins can be marked deprecated; clients receive warnings on install or use.
-

5. Discovery & Installation

CLI Commands:

List available plugins
janus plugin list

Search by keyword
janus plugin search <term>

Install a plugin
janus plugin install janus-plugin-schema-enhancer

Remove a plugin
janus plugin uninstall janus-plugin-schema-enhancer

•

Version Constraints: Users may pin major versions to ensure compatibility:

```
janus plugin install janus-plugin-bias-scanner==1.2.*
```

-
-

6. Security & Sandboxing

- **Isolated Environments:** Install plugins in a dedicated `plugins/` folder, import via `importlib` with limited `__builtins__`.
 - **Capability Restriction (future):** Define a plugin manifest to declare allowed actions (read-only vs mutation), enforced at load time.
 - **Auditing:** All plugin-invoked mutations are logged in the core audit log with plugin name and version.
-

7. Governance & Community

- **Code of Conduct:** Plugin authors agree to a community code of conduct.
 - **Support Channels:** Provide a forum or issue tracker for plugin support and bug reports.
 - **Certification:** Offer a “Janus Certified” badge for plugins that pass extended security and performance audits.
-

8. Next Steps

Thread #29 will detail a developer onboarding guide for writing, testing, and publishing a Janus plugin, including template repos and CI configurations.

End of Thread #28

Thread #29: Developer Onboarding Guide for Janus Plugins

This thread provides a step-by-step guide to help third-party developers write, test, and publish Janus plugins. It covers environment setup, code scaffolding, testing practices, CI configuration, packaging, and submission to the plugin registry.

1. Environment Setup

Clone the Janus repository:

```
git clone https://github.com/your-org/janus-cognitive.git
cd janus-cognitive
```

1.

Create a virtual environment and install core dependencies:

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -e .[dev]
```

2.

Verify CLI works:

```
janus --help
```

3.

2. Plugin Scaffolding

Generate plugin template:

```
mkdir -p janus_plugins/my_plugin
cat > janus_plugins/my_plugin/pyproject.toml << 'EOF'
[project]
name = "janus-plugin-my-plugin"
version = "0.1.0"
```



```
description = "My custom Janus plugin"
authors = ["Your Name <you@example.com>"]
dependencies = []
```

```
[project.entry-points.janus.plugins]
my_plugin = "my_plugin.process"
EOF
```

```
mkdir -p janus_plugins/my_plugin/my_plugin
touch janus_plugins/my_plugin/my_plugin/__init__.py
```

1.

Implement `process(engine, payload)`: in `my_plugin.py`:

```
def process(engine, payload):
    # TODO: mutate engine.G or compute metrics
    return {"graph_delta": "...", "output": "..."}

2.
```

3. Local Testing

Write pytest tests: Create `tests/test_my_plugin.py`:

```
from janus_core.engine import JanusGraphEngine
from janus_plugins.my_plugin.my_plugin import process
```

```
def test_process_runs_without_error():
    engine = JanusGraphEngine()
    result = process(engine, "sample payload")
    assert isinstance(result, dict)
```

1.

Run tests:

```
pytest tests/test_my_plugin.py
```

2.

4. CI Configuration

Extend GitHub Actions: in `.github/workflows/plugins.yml`:

```
name: Plugin CI
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with: python-version: '3.12'
      - run: pip install -e .[dev]
      - run: pytest tests/test_my_plugin.py
```

1.

5. Packaging & Publication

Build distribution:

```
cd janus_plugins/my_plugin
pip install build
python -m build
```

1.

Publish to PyPI or private index:

```
pip install twine
twine upload --repository pypi dist/*
```

2.

Tag a release:

```
git tag v0.1.0
git push origin v0.1.0
```

3.

6. Submission to Registry

Submit metadata: Use the CLI:

```
janus plugin submit --name janus-plugin-my-plugin --version 0.1.0 \
--url https://pypi.org/project/janus-plugin-my-plugin/0.1.0/ \
--description "My custom Janus plugin."
```

- 1.
 2. **Pass automated checks:** Registry CI will validate entry points and run smoke tests.
 3. **Approve & publish:** Core team reviews; upon approval, plugin appears in [janus plugin list](#).
-

7. Iteration & Maintenance

- **Version bumps:** Follow semantic versioning for backward compatibility.
 - **Deprecation:** Mark old versions as deprecated via registry metadata.
 - **Community Support:** Monitor issues, respond to bug reports, and update docs.
-

End of Thread #29

Thread #30: Community Support & Governance

This thread defines the structures, policies, and processes for community engagement, support, and governance around the Janus platform and its plugin ecosystem, ensuring healthy contributions, clear responsibilities, and sustainable growth.

1. Community Channels & Support

- **Discussion Forum/Chat:**
 - Host on platforms like GitHub Discussions, Discourse, or Slack/Discord.
 - Channels: #general, #plugins, #security, #performance, #help
 - **Issue Tracker:**
 - Centralized GitHub Issues for bug reports, feature requests, and support tickets.
 - Labels: bug, enhancement, question, plugin, security, performance
 - **Documentation Hub:**
 - docs/ directory in repo, served via GitHub Pages or ReadTheDocs.
 - Sections: Getting Started, API Reference, Plugins, Best Practices, FAQ.
 - **Office Hours / AMA:**
 - Regular community calls or livestream sessions to answer questions, roadmap updates, and gather feedback.
-

2. Code of Conduct

- **Purpose:** Foster an inclusive and respectful environment.
- **Key Principles:**
 - Be kind and considerate.
 - No harassment or discrimination.
 - Respect privacy and intellectual property.
- **Enforcement:**
 - Designated maintainers handle reports confidentially.

- Consequences: warning, temporary suspension, or ban.
 - **Location:** Publish at `CODE_OF_CONDUCT.md` in repo root.
-

3. Contribution Process

1. **Fork & Clone** the `janus-cognitive` repository.
 2. **Create a Feature Branch:** `git checkout -b feature/my-feature`
 3. **Implement & Test:**
 - Follow repository linting, formatting (e.g., `black`, `flake8`).
 - Add or update tests under `tests/`.
 4. **Pull Request (PR):**
 - Fill PR template with description, motivation, and screenshots (if any).
 - Tag appropriate labels and assign reviewers.
 5. **Review & Merge:**
 - Review by core maintainers.
 - Pass CI checks (tests, lint, type-check).
 - Merge into `main` and tag a new release if needed.
-

4. Plugin Contribution & Governance

- **Submission Guidelines:** See Thread #29 for onboarding.

- **Review Board:** A rotating group of core maintainers reviews plugin submissions for security and ethical compliance.
 - **Version Compatibility:** Plugins must declare compatibility via `Requires-Dist: janus-cognitive (>=X.Y,<X.Y+1)`.
 - **Deprecation Policy:**
 - Plugins that break compatibility or violate policies are marked deprecated in registry metadata.
 - Deprecation notice communicated via `janus plugin list --deprecated` and in the plugin registry UI.
-

5. Release Management & Roadmap Transparency

- **Release Cadence:** Quarterly minor releases (semver MINOR), patches as needed.
 - **Roadmap Publication:** Maintain `ROADMAP.md` with planned features, timelines, and milestones.
 - **Community Voting:** Periodic polls to prioritize features via GitHub Discussions.
-

6. Governance & Decision-Making

- **Steering Committee:** 3–5 elected maintainers overseeing project direction, releases, and code of conduct enforcement.
 - **RFC Process:** For major changes, use a `RFC/` directory where proposals are drafted, discussed, and approved before implementation.
 - **Transparency:** All governance meetings minutes published under `docs/governance/`.
-

7. Metrics & Health Indicators

- **Community Growth:** Number of contributors, active issues/PRs.
 - **Response Time:** Average time to first response on issues/PRs.
 - **Plugin Ecosystem:** Number of published plugins, downloads, compatibility coverage.
-

8. Next Steps

Thread #31 will design the **RFC template and process** for proposing and approving major Janus platform enhancements.

End of Thread #30

Thread #31: RFC Template and Approval Process

This thread defines the Request for Comments (RFC) procedure for proposing, discussing, and approving major enhancements to the Janus platform. It provides a standard template and governance workflow to ensure transparent, well-documented design decisions.

1. When to File an RFC

File an RFC for any change that:

- Introduces new core features or major architectural shifts.
- Alters symbolic metric definitions (CD, CM, PB) or safety thresholds.
- Changes the plugin architecture or governance model.
- Modifies data retention, archival, or security policies at scale.

Minor bug fixes, documentation updates, or small CLI enhancements do not require an RFC.

2. RFC Directory Structure

/janus

/RFC

XXXX-some-feature.md # e.g. 0001-new-metric-system.md

- Filenames begin with a zero-padded 4-digit number.
 - The number is assigned sequentially and referenced in commit messages.
-

3. RFC Template

Create a new file under `RFC/000N-short-title.md` using the template below.

RFC 000N: Short Title of Proposal

- **Authors:** Name1 <email1>, Name2 <email2>
- **Status:** Draft | Reviewing | Approved | Rejected
- **Created:** YYYY-MM-DD
- **Updated:** YYYY-MM-DD (if revised)

Abstract

A one-paragraph summary of the proposal.

Motivation

Explain the problem or limitation the change addresses.

Proposal

Describe the technical approach in detail, including:

- Diagrams or pseudocode for workflows or data models.
- Impacts on existing modules, APIs, and user workflows.

Drawbacks

List potential downsides or risks (e.g., performance, complexity).

Alternatives Considered

Outline other options explored and reasons for their rejection.

Compatibility

Specify version constraints, migration steps, and backward compatibility notes.

Unresolved Questions

Document any open issues requiring further discussion.

Plan

Proposed timeline, milestones, and testing strategy.

4. RFC Workflow

1. **Draft:** Author creates `RFC/000N-*.md` from the template and opens a PR against `main`.
2. **Announcement:** Post a summary in the community forum and label the PR `RFC`.
3. **Discussion & Revision:** Community and Steering Committee review, propose edits via PR comments or commits.
4. **Review Period:** Allow at least 7 days for community feedback.
5. **Decision:** Steering Committee updates **Status** to `Approved` or `Rejected` and merges (if approved) or closes (if rejected).
6. **Implementation:** If approved, authors open implementation PR(s) referencing the RFC.

5. Governance Roles

- **Authors:** Draft and shepherd the RFC through discussion.
- **Steering Committee:** Ensure alignment with project goals, arbitrate disputes, and make final decisions.
- **Community:** Provide feedback, propose alternatives, and vote (informally) on priorities.

6. Tracking & Documentation

- **Master List:** Add each RFC's metadata to [RFC/README.md](#) with links and status.
- **Versioning:** Reference RFC numbers in code comments and changelogs.
- **Archival:** Once implemented or rejected, mark as [Approved](#) or [Rejected](#) and move to [RFC/archive/](#) if desired.

7. Next Steps

Thread #32 will establish the **Changelog & Release Notes** guidelines, aligning new changes with semantic versioning and user communications.

End of Thread #31

Thread #32: Changelog & Release Notes Guidelines

This thread defines standards and best practices for maintaining a clear, informative changelog and publishing release notes for each Janus platform release, ensuring transparency and ease of adoption for users.

1. Changelog Format & Location

- **File:** [CHANGELOG.md](#) at repository root.
- **Structure:** Use [Keep a Changelog](#) format.
- **Sections:**

- **Unreleased** (top section for in-progress changes)
- **[X.Y.Z] - YYYY-MM-DD** for each released version, in descending order.

Example Entry

[1.2.0] - 2025-07-15

Added

- Support for `--strict` mode in CLI (Thread #4).
- New `projection\_bias` metric in Graph Engine (Thread #3).

Changed

- Updated plugin loader to use importlib (Thread #5).

Fixed

- Rollback snapshot timestamp ordering issue (Thread #4).

Deprecated

- YAML config support planned for 2.0.0.

Removed

- Legacy `echo\_stage` default registration.
-

2. Release Notes Publication

- **Tagging:** Create a Git tag **vX.Y.Z** matching the changelog entry.
- **GitHub Release:** Draft a GitHub Release using the changelog section for that version.
- **Templates:** Use a release template describing:
 - **Highlights:** Key features and improvements.
 - **Upgrade Instructions:** Any migration steps or breaking changes.
 - **Contributors:** List of PR authors and thank-yous.

Release Template

Janus Cognitive Recursion Platform vX.Y.Z

****Release Date:**** YYYY-MM-DD

Highlights

- Feature A: `_brief description_`
- Improvement B: `_brief description_`

Upgrade Instructions

1. ``pip install --upgrade janus-cognitive``
2. Update your ``janus_config.json`` with new thresholds.

Breaking Changes

- Configuration key ``old_key`` renamed to ``new_key``.

Contributors

Thanks to @alice, @bob, and @carol for their contributions!

3. Semantic Versioning Policy

- Follow [SemVer 2.0.0](#): MAJOR.MINOR.PATCH
 - **PATCH**: Bug fixes, non-breaking changes.
 - **MINOR**: New backward-compatible features (e.g., new plugin APIs).
 - **MAJOR**: Incompatible API changes or major architectural shifts.
-

4. Automation & CI Integration

- **Validation**: CI job to ensure `CHANGELOG.md` has an entry for the latest tag.
- **Release Script**: Provide `scripts/release.sh` to:
 1. Bump version in `pyproject.toml`.
 2. Append new header in `CHANGELOG.md` with date.
 3. Commit changes, tag, and push.

4. Draft GitHub Release via CLI (`gh release create`).

5. Communication & User Notification

- **Email Newsletter:** Summarize release highlights to subscribers.
- **Social Media:** Tweet or post on LinkedIn announcing major releases.
- **Documentation:** Update `docs/` with any new or changed features.

End of Thread #32